

Scratch and Sense: Using Real-Time Sensor Data to Motivate Students Learning Scratch

Hussel Suriyaarachchi
Augmented Human Lab
The University of Auckland
Auckland, New Zealand
hussel@ahlab.org

Paul Denny
School of Computer Science
The University of Auckland
Auckland, New Zealand
paul@cs.auckland.ac.nz

Suranga Nanayakkara
Augmented Human Lab
The University of Auckland
Auckland, New Zealand
suranga@ahlab.org

ABSTRACT

Block-based programming environments are a popular way to introduce programming as they provide helpful visual cues and remove the complexities of syntax, allowing learners to focus on being creative. However, input to programs in block-based environments is often limited to the keyboard and mouse, meaning programs respond only to the direct actions of the user. Allowing programs to respond to changes in the physical environment may influence the types of programs students create and their motivation and interest towards learning programming. We explore this idea by integrating real-time sensor data into Scratch. With our platform, students simply connect a sensor to their computer via USB and use custom blocks, alongside conventional Scratch code, to read and react to the sensor data in real-time. We evaluated this approach using a field study involving 25 students with limited prior experience in Scratch. We found students were highly motivated by the sensors and expressed a strong desire to use them in future projects. Analysis of the created programs revealed that, compared to using standard input sources, students explored a wider variety of Scratch blocks when controlling sprites using the sensors.

CCS CONCEPTS

• **Social and professional topics** → *Computing education*.

KEYWORDS

Scratch, Sensors, Real-time, Motivation, Blocks, Kiwriious, Physical Computing

ACM Reference Format:

Hussel Suriyaarachchi, Paul Denny, and Suranga Nanayakkara. 2022. Scratch and Sense: Using Real-Time Sensor Data to Motivate Students Learning Scratch. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2022)*, March 3–5, 2022, Providence, RI, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3478431.3499316>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGCSE 2022, March 3–5, 2022, Providence, RI, USA

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9070-5/22/03...\$15.00
<https://doi.org/10.1145/3478431.3499316>

1 INTRODUCTION

As computing becomes increasingly recognised as an essential skill for people of all ages, so too does the use of block-based environments for introducing learners to the practice of programming [33]. Particularly for young children, block-based programming environments provide helpful visual cues and avoid the difficulties related to syntax errors which can be frustrating for students using text-based approaches [4]. Popular platforms such as Scratch [28], Snap! [20] and Blockly [17] are now in widespread use around the world.

Motivation is a particularly critical ingredient for effective learning, as motivated students demonstrate greater persistence and engagement in academic tasks and therefore achieve better long-term outcomes [13, 15, 26]. It is only natural then, that computing educators are interested in effective ways of fostering student motivation in the classroom [14]. Intrinsic motivation, in particular, is known to have a very positive impact on programming outcomes [6, 7].

Constructivist learning theories [5] have inspired physical computing pedagogies for teaching programming. The ability to control physical systems that sense their environment can increase both motivation and creativity in students learning to program [30]. Tangible devices with built-in sensors and actuators are ideal for motivating young children to learn computing as they can be programmed using block-based environments. Initiatives such as the BBC micro:bit [30] and Lego Mindstorms [22] are popular platforms for creating programs that control hardware (actuation). Conversely, hardware can be used to control how programs behave (sensing). Input to programs in block-based settings is often limited to the keyboard and mouse, meaning programs react only to direct user actions. The ability to sense physical phenomena for use as input to block-based programs may impact the motivation and creativity of young learners. Devices like the micro:bit [30] and Arduino [25] provide sensing capabilities, but offer a limited range of sensors by default, require manual wiring to add new sensors, and need to be flashed, or have code transferred to them, prior to use.

In this work, we present and evaluate *Scratch and Sense*, a novel extension to the Scratch platform that seamlessly integrates an affordable sensor kit enabling the real-time measurement of a wide variety of environmental data. Using *Scratch and Sense*, students simply connect a sensor via USB and can then immediately begin writing Scratch programs that react to the sensor values. *Scratch and Sense* addresses some of the limitations of existing sensor toolkits. The sensors are affordable, measure a variety of data including air quality and conductance, and require no wiring or manual configuration. We report the results of a pilot study conducted in a classroom setting involving 25 students with limited prior experience of Scratch, guided by the following questions:

- (1) To what extent are students motivated by real-time sensor data when learning to program in Scratch?
- (2) Do students engage with a wider variety of Scratch blocks when controlling sprites with sensor input, compared to more traditional inputs?

2 BACKGROUND

Since the early 1990s, a wide array of hardware-based tools such as robots, programmable toys and microcontrollers have made their way into science classrooms to provide engaging experiences for learning computational thinking. However, programming such devices was not always straightforward for young children, particularly when manual wiring or text-based programming was required [16]. As block-based programming environments have grown in popularity and proven to be appropriate for young learners, they are now commonly used for programming hardware-based educational tools [27]. Most tools currently in use focus the learners' attention on writing programs that control a physical device on which code execution is observed [22, 25, 30]. Much less attention has been paid to the use of hardware for providing real-time sensor input to control the behavior of executing programs.

2.1 Block Programming Environments

Block-based programming environments that run in a web-browser provide an easily accessible, device-agnostic approach to coding. Adapting such environments to accommodate hardware input usually involves installing an offline software component to support the extended functionality. This is evident in popular block-based platforms such as Scratch [28] and Snap! [20], where supported devices must initially be set up through utility software^{1,2}.

To write a program in a block-based environment that makes use of hardware-based sensor input, appropriate blocks must be available in the language for reading sensor data. Popular environments such as Scratch do not provide suitable blocks by default, leading to the development of custom environments that are modifications of Scratch. For example, mBlock by Makeblock is an online tool supporting interactivity with a range of technologies [16], and is commonly used for programming robotic kits such as the Arduino and micro:bit. Heavily inspired by Scratch, mBlock provides users with an almost identical programming interface. RobboScratch³ and Pictoblox⁴ are similar offline alternatives. While these solutions provide superior integration with hardware compared to the official Scratch platform, they introduce the dependency of requiring custom software.

2.2 Sensor Input in Block Programming

General-purpose microcontrollers are versatile electronic tools that can be purposed as input to a program by connecting a wide range of sensors. Enabling sensor connectivity for this genre of devices often requires knowledge of wiring, pin configuration and serial communication. Moreover, the desired sensors usually need to be purchased separately from the microcontroller. These concerns

have been explored, for example in research by Booth and Stumpf [8] and by Heo [21], in the context of the Arduino [25] which is a popular microcontroller used in STEM education and compatible with block-based platforms.

Many of these shortcomings were addressed with the introduction of the BBC micro:bit [3, 30] in 2015. The micro:bit is an open source hardware system designed specifically for educational use, initially released in computing education classrooms in the United Kingdom. It is a microcontroller-based device that can be leveraged as an input mechanism for block-based programs. It contains a selection of built-in sensors that offer improved accessibility when programming while reserving the option for sensor augmentations [18]. However, there remain technical pitfalls for classroom deployment, as highlighted by the experiences of Tyrén et al. [31], including its need to be flashed for programs to work. Pairing via Bluetooth was another documented challenge, with the authors warning instructors against asking students to imitate the procedure as it “can lead to disorder in the class”. Finally, just as *ease of use* is an important concern, so is *depth of use* as described by Austin et al. [3] in their review of the micro:bit project's history. While the onboard sensors of the micro:bit are sufficient for general use, a wider variety of environmental sensors could inspire students to explore their ideas more deeply and construct programs of greater complexity.

Other types of programmable hardware could be used to support real-time sensor input to programs, but introduce unnecessary cost or complexity. McNally et al. [24] discuss the infeasibility of providing individual Lego Mindstorm [22] kits to a classroom of students for cost reasons. In addition, they note the absence of a rapid feedback loop which would enable students to conveniently test, modify and reflect on changes made to their programs. Due to the build-first approach employed by Lego Mindstorms, the need to assemble Lego bricks and sensors for use as real-time input devices may draw attention away from the coding task at hand [19]. CyberPi⁵, a single-board computer developed by Makeblock, presents a similar drawback with the requirement for sensors to be attached to the device when programming. Using intermediary devices to enable sensor connectivity with a block-based programming environment appears to be an unnecessary overhead.

2.3 Goals for *Scratch and Sense*

In developing *Scratch and Sense*, we chose to target Scratch as it is currently the most widely used block-based programming platform [32]. Although in the current work we focus on helping young children learn programming, prior literature involving Scratch has shown it is appropriate for novice and advanced learners alike [2, 32, 34].

After observing the limitations of existing approaches for supporting real-time sensor input into block-based programs, our goal is to provide a solution which: (1) is very easy for students to use (plug-and-play); (2) is highly accessible (in terms of cost and device support); (3) is easily extensible to support a wide range of new sensors.

¹https://en.scratch-wiki.info/wiki/Scratch_Link

²<http://snap4arduino.rocks/>

³<https://www.robbo.world/apps/>

⁴<https://thetempedia.com/tutorials/getting-started-pictoblox/>

⁵<https://education.makeblock.com/cyberpi-go-kit/>

3 DESIGN

3.1 Sensor Technology

We used an affordable off-the-shelf sensor kit from Kiwriious⁶ for our implementation. The kit comprises six sensors that plug directly into a USB port, and measure various physical phenomena including humidity, ambient temperature, total volatile organic compounds, UV light and visible light and conductance. Our motivation to incorporate such a range of natural phenomena into a block-based programming environment was inspired by recent work that has made use of sensor technologies for supporting learning in other contexts and domains [10, 18].

We leverage the Web Serial API offered in Chromium-based browsers to communicate with the sensors via the serial port. The API provides an asynchronous communication interface for receiving raw byte data from the sensors. This avoids the need for installing further software dependencies, in line with our accessibility goals. The combination of Scratch and our chosen sensor technology provides a device-independent way of programming with sensor data through the browser.

3.2 Scratch Extension

We developed *Scratch and Sense* as a Scratch Extension. Scratch Extensions⁷ are an accepted approach for introducing external hardware and additional functionality into Scratch. Through extensions, custom blocks can be loaded into the Scratch editor with the click of a button, and then used alongside standard Scratch blocks to create programs.

Following the technical documentation⁸ provided by Scratch, we created an extension defining all the blocks necessary for the configuration, retrieval and manipulation of sensor data. This extension was made accessible for end-users by deploying a web application running a clone of the Scratch environment where its Virtual Machine (VM) was modified to include a new JavaScript class. Figure 1 illustrates the complete collection of blocks that we developed for our custom extension. These blocks are classified into two types: ‘Connect and Read Forever’ blocks which establish the connection to the sensors and are used once in every program, and ‘Sensor’ blocks which provide real-time access to read sensor data.

Connect and Read Forever Blocks: *Connect* and *Read Forever* are command blocks in Scratch. The *Connect* block establishes a USB serial connection, and the *Read Forever* block starts a thread that continuously receives and decodes sensor data. In all *Scratch and Sense* projects, users need to connect a Start Block (i.e. when ‘green flag clicked’ block) with the *Connect* and *Read Forever* blocks (as illustrated in Figure 1A). These three blocks, together, operate all tasks required for setting up and maintaining the sensor connection and communication.

Sensor (Variable) Blocks: Sensor blocks are reporter blocks (i.e. blocks that report a value) in Scratch. These blocks allow users to access the specific sensor data. In *Scratch and Sense* we have currently implemented eight Sensor blocks (See Figures 1C-1F) for

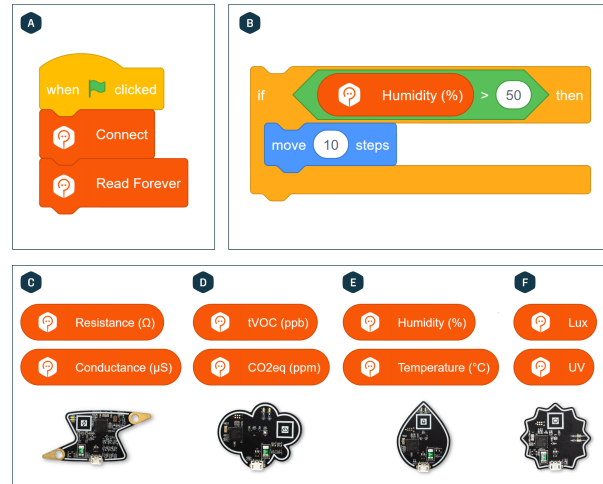


Figure 1: A) Connecting to sensors; B) Using sensor data in Scratch; Measuring C) conductance, D) air quality, E) humidity and ambient temperature, F) UV and visible light.

reading eight varieties of physical data. A user can simply drag-and-drop these Sensor blocks to connect them with standard Scratch blocks. For example, as shown in Figure 1B, the Humidity Sensor block has been placed within a conditional block to move a sprite by 10 steps if the humidity reading is greater than 50%.

4 EVALUATION

4.1 Participants

We evaluated *Scratch and Sense* in a study with 25 students (14 boys, 11 girls) aged between 11 to 13 years from a public intermediate school. The students were involved in a digital technologies class that introduced them to programming with Scratch. At the time of the study, they were 14 weeks (14 hours) into their syllabus. Students were also familiar with Makey Makey [12], an electronic learning kit enabling computer interaction through tangible interfaces. Makey Makey was taught in conjunction with the Scratch lessons and accounted for half of the total teaching time. While the students already had some prior exposure to digital technologies, our study was the first time they worked with sensors that provided plug-and-play access to environmental data.

4.2 Procedure

The study was conducted within a classroom during school time. It consisted of a 15-minute introduction, 1-hour open-ended Scratch coding activity and a 15-minute feedback session.

4.2.1 Introduction and starter program (15 mins). As illustrated in Figure 2, our introduction demonstrated a simple starter Scratch project that explored different ways of providing input to Scratch. Figure 2A (top) shows the typical approach of handling a keypress by placing sensing logic inside a conditional block. A collection of Scratch blocks were connected to create the script shown in Figure 2C that moves the sprite ten steps to the right when the right arrow key on the keyboard is pressed.

We then presented each sensor of the toolkit to the class and explained what it could measure. Using the humidity sensor, the starter project was extended to include a new sensor-controlled

⁶<https://kiwriious.com/>

⁷https://en.scratch-wiki.info/wiki/Scratch_Extension

⁸<https://github.com/LLK/scratch-vm/blob/develop/docs/extensions.md>

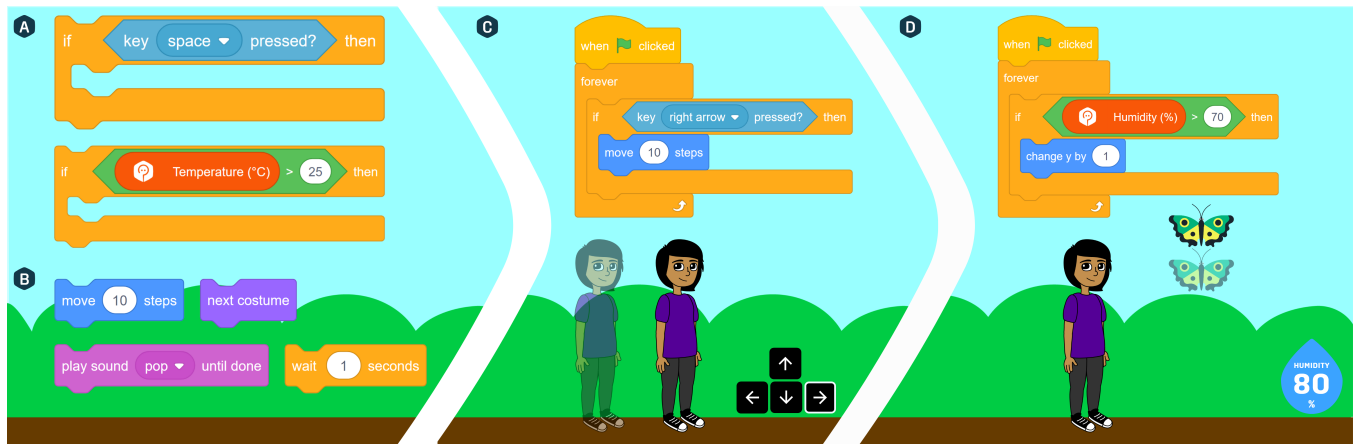


Figure 2: A) Obtaining input from non-sensor (top) and sensor (bottom) sources; B) Example Scratch blocks that can be placed inside (A) and run when condition is satisfied; C) Starter project with keyboard-controlled sprite that moves right when right arrow key is pressed; D) Sensor-controlled sprite where butterfly moves upwards when humidity is greater than 70%.

sprite. Figure 2D shows the script used in moving this new sprite upwards when the humidity level rises over 70%.

As seen in Figure 2A, alternating from keyboard input to sensor input simply involves replacing a single logical operator inside the conditional block. Thus, by creating an example program consisting of two sprites with similar behaviour, we highlighted to students the near-identical processes involved to use either the keyboard or the sensors as input mechanisms for their Scratch programs.

4.2.2 Free exploration (60 mins). The introduction was followed by a 60 minute free-exploration period, guided by the Use-Modify-Create pedagogy [23, 29]. Students were encouraged to download the example project, explore its code and make modifications to it, or to create a brand new program. A supplementary website⁹ was created to provide basic reference information on Scratch concepts and sensor usage. Sensor kits were made available for students to use with the school-designated computers, however students were not required to use the sensors in their programs. It was made clear to students that they were free to create any program, using any kind of input, during this free exploration period. Several researchers and the class teacher, all of whom were proficient with Scratch, were present for the duration of the session to offer assistance and take observational notes.

We tested and verified that our extension worked reliably with the sensor toolkit across various platforms and computers through a small scale pilot deployment four weeks prior to this study. This earlier pilot followed an identical procedure and was conducted in a classroom of five students at a private boys school.

4.2.3 Program submission and feedback session (15 mins). At the end of the free exploration period, students were asked to submit the program they had created using a custom sharing feature that uploaded the program to a Padlet board¹⁰. A paper-based questionnaire containing a mixture of Likert and open-ended questions was then distributed. This questionnaire contained the interest/enjoyment subscale from the Intrinsic Motivation Inventory (IMI) [1], which is considered the self-report measure of intrinsic

motivation. One additional question was included, asking students their thoughts on using the sensors in future projects. Students responded to the questions on a 7-point Likert scale. The questions, and a summary of the data collected, are shown in Table 1.

The following three open-ended questions were also included on the questionnaire to gather insights into students' perceptions of programming with sensors and the challenges they faced: (1) "Write down an interesting thing that you did in your project and how you made it happen"; (2) "What were the challenges you faced when making your project?"; and (3) "What made you want to use the sensors? If you did not use the sensors, tell us why".

4.3 Data Analysis

We received submissions from 21 students, each containing a Scratch project and a completed questionnaire. Thematic coding [9] was performed on the open-response feedback to organise the responses around common themes related to motivation. These findings were further supported by our observational notes, and quantitative data from responses to the questionnaire. Finally, static analysis of the submitted Scratch projects was performed to classify the variety of blocks used in the students' programs.

As programming in Scratch usually occurs within sprites [2], our initial program analysis task was to determine which sprites were sensor-controlled and which were not. Scratch formally defines a *script* as a collection of connected blocks that determine how a sprite behaves (Figures 2C and 2D are examples of scripts). We examined the scripts that were associated with each sprite.

We performed a binary classification of every script based on whether it contained one of our custom sensor blocks or a standard sensing (e.g. keyboard input) block (see Figure 2A). In our analysis that follows, we refer to such scripts as *sensor-based* or *non-sensor based* respectively. Scripts that did not contain either type of block were excluded from further analysis. Although there was potential for scripts to contain both sensor and sensing blocks, none were observed.

We then inspected the individual blocks used in creating each script. Since we were interested in investigating the variety of Scratch blocks that students engaged with, any blocks required to be

⁹<https://sites.google.com/ahlab.org/kiwrious-play/kiwrious-getting-started>

¹⁰<https://padlet.com/KiwriousPlay/49fmsyic2yhrfr1>

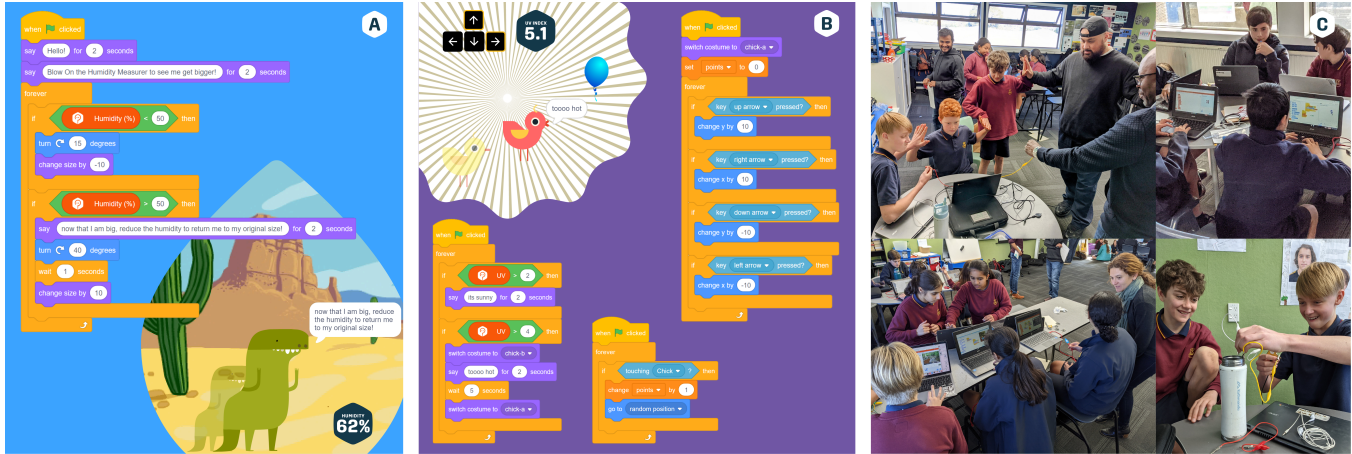


Figure 3: A) Student-created program investigating the humidity of breath, where blowing on the humidity sensor affects the size of the dinosaur; B) Student-created game using the UV and light sensor, where the bird is moved via arrow keys to chase a balloon and changes colour and displays messages according to UV readings; C) Students engaging in free exploration.

used by default were excluded (such as Start Blocks). All remaining blocks were those optionally chosen by students to control the sprites in their programs (see Figure 2B). Categorising Scratch blocks into groups has been used in prior exploratory studies [2], and we adopt a similar approach here. We use the default block grouping defined by Scratch and we calculate the frequency of use for blocks of each category. Figure 4 shows the breakdown of blocks of each type between sensor-based and non-sensor based scripts.

5 RESULTS & DISCUSSION

5.1 Motivation for Learning

Despite having the choice of whether or not to use the sensors during the free exploration period, all students created programs that used the sensors. Seventeen projects were strictly sensor-based, and four projects used both sensor-based and standard input.

Thematic Analysis of Open Responses. Three main themes emerged from our analysis: interest, interactivity and collaboration.

Students described their experience of programming with sensors in Scratch as “fun”, and “interesting” (17 comments). They expressed interest in creating programs that investigated the different physical phenomena (4 comments). One student stated that they wanted “to know what the job of the sensors was and how much are its average readings”. The program shown in Figure 3A illustrates one student’s exploration of humidity, where the moisture of their breath was used to control a sprite by blowing onto the sensor.

The ability to connect with the environment created an interactive programming experience. Students who used the light sensor experimented with torches and walked around the classroom to investigate the behaviour of their program at different light levels, “I made my character move once I put the torch on the light sensor”. The student project illustrated in Figure 3B is a good example of this, as the student would need to move the sensor in and out of direct sunlight or use a UV torch to control the sprite’s appearance. We also observed students connecting the Conductance sensor to various objects around the classroom. Everyday objects like bottles, stationery and clothing were used to interact with the programs

created (see Figure 3C). One student writes: “I connected the conductance sensor to my pants, which had metal in it.”

The sensors, being a novel technology for the students, provided very good opportunities for collaborative learning. We observed students often seeking help from their peers to solve programming problems rather than requesting assistance from the teacher. They appeared excited to showcase their work and ideas with their friends: “It looked really cool, and seeing my friends use it (sensors) made me feel like using it”. Some programs were designed to be used with collaboration in mind. A particularly interesting example of such a project used the Conductance sensor and relied on students holding hands for the program to function (see Figure 3C): “We made cool contraptions, and we made a human chain, and the current still worked”. This collaboration was also noticed by the teacher, who said during an interview after the session: “What’s good about using Scratch and Sense is that students are curious. They think about what the sensors measure and what they can do using them. The kids will then liaise with each other and will try making new stuff.”

Finally, some students experienced challenges when programming and conveyed their frustrations in finding the right blocks (5 comments): “I think the challenges I faced were what blocks I should use.” Incorrect code execution (7 comments) was another problem faced: “It was hard to get everything to do what I wanted it to do”. However, most of these issues were related to a lack of knowledge in Scratch rather than being specific to our tool. Although Scratch resources and other help were available, most students preferred to figure things out independently or with their friends.

Quantitative Survey Results. As all students chose to use the sensors in their programs, the responses to the IMI questions reflected their motivations for sensor use. The Likert scores in Table 1 showed a left-skewed distribution across all subscale items (with distributions being reversed for negative questions). Responses from students who encountered difficulties during the activity were recognised as having less positive Likert scores. The reported subscale score of 5.96 ($SD = 1.00$) on a scale of 7 suggests a favourable outlook on enjoyment, fun and interest. In prior work, Carbonell et al. [11] used IMI subscales in a similar context to analyse student motivation

Table 1: Questionnaire responses to the Intrinsic Motivation Inventory (IMI) interest/enjoyment subscale and preference for future sensor use. The suffix (R) represents negatively worded-questions for which scores must be reversed. Responses on 7-point Likert scale: NT: Not true at all (1), ST: Somewhat true (4), VT: Very true (7). Most popular response underlined.

Question		NT	2	3	ST	5	6	VT
IMI ₁	I enjoyed doing this activity very much	0%	0%	0%	9%	29%	19%	<u>43%</u>
IMI ₂	This activity was fun to do	0%	0%	0%	10%	24%	14%	<u>52%</u>
IMI ₃	I thought this was a boring activity (R)	<u>48%</u>	38%	0%	0%	5%	9%	0%
IMI ₄	This activity did not hold my attention at all (R)	<u>62%</u>	9%	19%	5%	5%	0%	0%
IMI ₅	I would describe this activity as very interesting	0%	0%	5%	9%	19%	29%	<u>38%</u>
IMI ₆	I thought this activity was quite enjoyable	0%	0%	0%	5%	14%	29%	<u>52%</u>
IMI ₇	While I was doing this activity, I was thinking about how much I enjoyed it	0%	0%	9%	19%	10%	<u>48%</u>	14%
Q ₁	I would like to use the sensors in my future Scratch projects	0%	0%	0%	9%	5%	24%	<u>62%</u>

for using a virtual reality tool to develop spatial knowledge. They considered an overall subscale score above the neutral response of 4 to be satisfactory. In comparison, our IMI data indicates that students were highly intrinsically motivated by the sensors.

Students expressed great enthusiasm to continue using sensors beyond our classroom study with requests to take the sensors home or leave them in the classroom (we did leave around 15 classroom sets). The self-report on future sensor use supported these statements with a highly positive agreeability score (91%) for wanting to use the sensors in future projects.

5.2 Variety of Blocks Used

Students demonstrated extensive use of sensor-based input. Across the 21 submitted projects (56 total scripts), 47 scripts used sensor-based input while only 9 scripts used standard input. Figure 4 illustrates the variety and frequency of blocks (organised into standard Scratch block categories) used for controlling sprites in sensor-based and non-sensor based scripts. A total of 138 individual blocks were used in all of the scripts, and 115 of these blocks were used in sensor-based scripts. The distribution suggests that students explored a broader range of Scratch blocks when using the sensors.

The most commonly used blocks were *Motion* and *Looks* blocks, suggesting that the animation of sprites was a popular response to sensor data. For example, animation featured in the project shown in Figure 3A by using blue *Motion* blocks and purple *Looks* blocks to

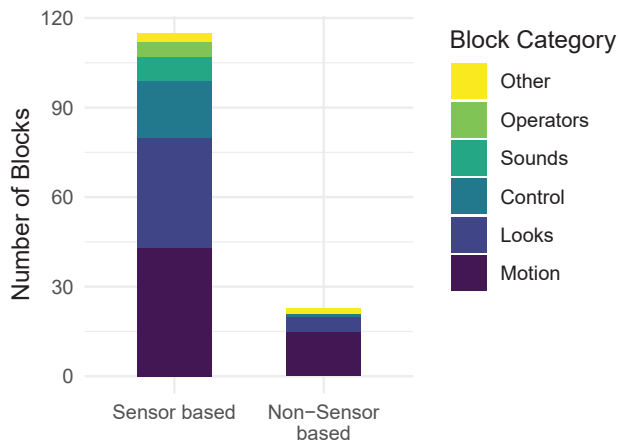


Figure 4: The frequency and variety of blocks used for controlling sprites in sensor-based and non-sensor based scripts. Standard Scratch block categories shown ('Other' includes rarely used Events, Sensing and Variable blocks).

rotate and change the sprite's size, respectively. The sensor-based scripts found in Figures 3A and 3B illustrate the use of 'wait' blocks, a type of Scratch *Control* block for introducing delays. Such blocks were used in moderation in the sensor-based scripts, to synchronise sprite movement. Occasional use of *Operators* blocks that offer further complexity to a sprite's actions were found inside *Motion* and *Control* blocks that accommodate data entry. For instance, an *Operator* block could replace the angle value shown in the blue *Motion* rotate block of Figure 3A with a calculation.

The keyboard was the only input device used in non-sensor based scripts, with keypresses invoking subsequent actions in sprites. These scripts primarily used *Motion* and *Looks* blocks for enacting sprite control. As seen in the non-sensor based script of Figure 3B, most of the blocks used in controlling the sprite were blue *Motion* blocks. While *Control* and *Other* blocks were rare, blocks from *Sounds* and *Operators* were absent across all non-sensor scripts.

5.3 Limitations

Although quantitative and qualitative measures agree in supporting students' motivations for using *Scratch and Sense*, our positive findings may have been influenced by a novelty bias – due to using the sensors for the first time. We attempted to mitigate this in the design of our classroom evaluation. The hour-long free exploration activity allowed students time to experiment with the sensors, if desired, and then decide whether to pursue creating a program that utilised them. Novelty effects, at least very short-lived ones, may have subsided before the end of the session and before students submitted their programs. Nevertheless, a longitudinal analysis of the use of the sensors is warranted.

6 CONCLUSION

We describe the development and evaluation of *Scratch and Sense*, a tool that enables the measurement of real-time environmental data for use in Scratch programs through the seamless integration of plug-and-play sensors. In a classroom evaluation, we found that students were highly motivated to program using the sensors and expressed a strong desire to use them in future projects. Compared to using standard input sources such as the keyboard, students also explored a wider variety of Scratch blocks when programming with the sensor data. Given the enthusiasm with which students made use of the easily accessible sensor data, and the robust relationship between motivation and learning, there may be great value in exploring closer integration between a broader array of plug-and-play sensors and block-based programming interfaces such as Scratch.

ACKNOWLEDGMENTS

This work was supported by the Assistive Augmentation research grant under the Entrepreneurial Universities (EU) initiative of New Zealand. We would like to thank the Kiwrious team, King's School and Kowhai School New Zealand for their support in making this work possible.

REFERENCES

- [1] [n.d.]. Intrinsic motivation inventory (IMI). <https://selfdeterminationtheory.org/intrinsic-motivation-inventory/>. Accessed: 2021-05-27.
- [2] Efthimia Aivaloglou and Felienne Hermans. 2016. How Kids Code and How We Know: An Exploratory Study on the Scratch Repository. In *Proceedings of the 2016 ACM Conference on International Computing Education Research* (Melbourne, VIC, Australia) (ICER '16). Association for Computing Machinery, New York, NY, USA, 53–61. <https://doi.org/10.1145/2960310.2960325>
- [3] Jonny Austin, Howard Baker, Thomas Ball, James Devine, Joe Finney, Peli De Halleux, Steve Hodges, Michal Moskal, and Gareth Stockdale. 2020. The BBC Micro:Bit: From the U.K. to the World. *Commun. ACM* 63, 3 (feb 2020), 62–69. <https://doi.org/10.1145/3368856>
- [4] Brett A. Becker, Paul Denny, Raymond Pettit, Durell Bouchard, Dennis J. Bouvier, Brian Harrington, Amir Kamil, Amey Karkare, Chris McDonald, Peter-Michael Osera, Janice L. Pearce, and James Prather. 2019. Compiler Error Messages Considered Unhelpful: The Landscape of Text-Based Programming Error Message Research. In *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education* (Aberdeen, Scotland UK) (ITICSE-WGR '19). Association for Computing Machinery, New York, NY, USA, 177–210. <https://doi.org/10.1145/3344429.3372508>
- [5] Mordechai Ben-Ari. 1998. Constructivism in Computer Science Education. In *Proceedings of the Twenty-Ninth SIGCSE Technical Symposium on Computer Science Education* (Atlanta, Georgia, USA) (SIGCSE '98). Association for Computing Machinery, New York, NY, USA, 257–261. <https://doi.org/10.1145/273133.274308>
- [6] Susan Bergin, Ronan Reilly, and Desmond Traynor. 2005. Examining the Role of Self-Regulated Learning on Introductory Programming Performance. In *Proceedings of the First International Workshop on Computing Education Research* (Seattle, WA, USA) (ICER '05). Association for Computing Machinery, New York, NY, USA, 81–86. <https://doi.org/10.1145/1089786.1089794>
- [7] Susan Bergin and Ronan G. Reilly. 2005. The Influence of Motivation and Comfort-Level on Learning to Program. In *PIPG. Psychology of Programming Interest Group*.
- [8] Tracey Booth and Simone Stumpf. 2013. End-User Experiences of Visual and Textual Programming Environments for Arduino. In *End-User Development*, Yvonne Dittrich, Margaret Burnett, Anders Mørch, and David Redmiles (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 25–39. https://doi.org/10.1007/978-3-642-38706-7_4
- [9] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. <https://doi.org/10.1191/1478088706qp0630a>
- [10] Jiashuo Cao, Samantha W. T. Chan, Dawn L. Garbett, Paul Denny, Alaeddin Nassani, Philipp M. Scholl, and Suranga Nanayakkara. 2021. Sensor-Based Interactive Worksheets to Support Guided Scientific Inquiry. In *Interaction Design and Children* (Athens, Greece) (IDC '21). Association for Computing Machinery, New York, NY, USA, 1–7. <https://doi.org/10.1145/3459990.3460716>
- [11] Carlos Carbonell-Carrera and Jose Luis Saorin. 2017. Geospatial Google Street View with Virtual Reality: A Motivational Approach for Spatial Training Education. *ISPRS International Journal of Geo-Information* 6, 9 (2017), 261. <https://doi.org/10.3390/ijgi6090261>
- [12] Beginner's Mind Collective and David Shaw. 2012. Makey Makey: Improvising Tangible and Nature-Based User Interfaces. In *Proceedings of the Sixth International Conference on Tangible, Embedded and Embodied Interaction* (Kingston, Ontario, Canada) (TEI '12). Association for Computing Machinery, New York, NY, USA, 367–370. <https://doi.org/10.1145/2148131.2148219>
- [13] Terence J Crooks. 1988. The Impact of Classroom Evaluation Practices on Students. *Review of Educational Research* 58, 4 (1988), 438–481. <https://doi.org/10.3102/00346543058004438>
- [14] Paul Denny, Brett A. Becker, Michelle Craig, Greg Wilson, and Piotr Banaszkiewicz. 2019. Research This! Questions That Computing Educators Most Want Computing Education Researchers to Answer. In *Proceedings of the 2019 ACM Conference on International Computing Education Research* (Toronto ON, Canada) (ICER '19). Association for Computing Machinery, New York, NY, USA, 259–267. <https://doi.org/10.1145/3291279.3339402>
- [15] Carol S Dweck. 1986. Motivational processes affecting learning. *American Psychologist* 41, 10 (1986), 1040–1048. <https://doi.org/10.1037/0003-066X.41.10.1040>
- [16] Aamir Fidai, Mary Margaret Capraro, and Robert M. Capraro. 2020. "Scratch"-ing computational thinking with Arduino: A meta-analysis. *Thinking Skills and Creativity* 38 (2020), 100726. <https://doi.org/10.1016/j.tsc.2020.100726>
- [17] Neil Fraser. 2015. Ten Things We've Learned from Blockly. In *Proceedings of the 2015 IEEE Blocks and Beyond Workshop (Blocks and Beyond) (BLOCKS AND BEYOND '15)*. IEEE Computer Society, USA, 49–50. <https://doi.org/10.1109/BLOCKS.2015.7369000>
- [18] Alexandra Gendreau Chakarov, Quentin Biddy, Jennifer Jacobs, Mimi Recker, and Tamara Sumner. 2020. Opening the Black Box: Investigating Student Understanding of Data Displays Using Programmable Sensor Technology. In *Proceedings of the 2020 ACM Conference on International Computing Education Research* (Virtual Event, New Zealand) (ICER '20). Association for Computing Machinery, New York, NY, USA, 291–301. <https://doi.org/10.1145/3372782.3406268>
- [19] Valentin Haak, Joerg Abke, and Kai Borgeest. 2018. Conception of a Lego Mindstorms EV3 simulation for teaching C in computer science courses. In *2018 IEEE Global Engineering Education Conference (EDUCON)*. 478–483. <https://doi.org/10.1109/EDUCON.2018.8363269>
- [20] Brian Harvey and Jens Mönig. 2010. Bringing "no ceiling" to scratch: Can one language serve kids and computer scientists. *Proc. Constructionism* (2010), 1–10.
- [21] Gyeongyong Heo. 2019. Implementation of an Arduino Compatible Modular Kit for Educational Purpose. *Journal of the Korea Institute of Information and Communication Engineering* 23, 5 (2019), 547–554. <https://doi.org/10.6109/jkiice.2019.23.5.547>
- [22] F. Klassner and S.D. Anderson. 2003. LEGO MindStorms: not just for K-12 anymore. *IEEE Robotics Automation Magazine* 10, 2 (2003), 12–18. <https://doi.org/10.1109/MRA.2003.1213611>
- [23] Irene Lee, Fred Martin, Jill Denner, Bob Coulter, Walter Allan, Jeri Erickson, Joyce Malyn-Smith, and Linda Werner. 2011. Computational Thinking for Youth in Practice. *ACM Inroads* 2, 1 (feb 2011), 32–37. <https://doi.org/10.1145/1929887.1929902>
- [24] Myles McNally, Michael Goldweber, Barry Fagin, and Frank Klassner. 2006. Do Lego Mindstorms Robots Have a Future in CS Education?. In *Proceedings of the 37th SIGCSE Technical Symposium on Computer Science Education* (Houston, Texas, USA) (SIGCSE '06). Association for Computing Machinery, New York, NY, USA, 61–62. <https://doi.org/10.1145/1121341.1121362>
- [25] David Mellis, Massimo Banzi, David Cuartielles, and Tom Igoe. 2007. Arduino: An open electronic prototyping platform. In *Proc. Chi*, Vol. 2007. 1–11.
- [26] Paul R Pintrich and Elisabeth V De Groot. 1990. Motivational and self-regulated learning components of classroom academic performance. *Journal of Educational Psychology* 82, 1 (1990), 33–40. <https://doi.org/10.1037/0022-0663.82.1.33>
- [27] Adin Baskoro Pratomo and Riza Satria Perdana. 2017. Arduviz, a visual programming IDE for arduino. In *2017 International Conference on Data and Software Engineering (ICoDSE)*. IEEE Computer Society, USA, 1–6. <https://doi.org/10.1109/ICoDSE.2017.8285871>
- [28] Mitchel Resnick, John Maloney, Andrés Monroy-Hernández, Natalie Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman, and Yasmin Kafai. 2009. Scratch: Programming for All. *Commun. ACM* 52, 11 (nov 2009), 60–67. <https://doi.org/10.1145/1592761.1592779>
- [29] Jean Salac, Cathy Thomas, Chloe Butler, Ashley Sanchez, and Diana Franklin. 2020. TIPP&SEE: A Learning Strategy to Guide Students through Use - Modify Scratch Activities. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education* (Portland, OR, USA) (SIGCSE '20). Association for Computing Machinery, New York, NY, USA, 79–85. <https://doi.org/10.1145/3328778.3366821>
- [30] Sue Sentance, Jane Waite, Steve Hodges, Emily MacLeod, and Lucy Yeomans. 2017. "Creating Cool Stuff": Pupils' Experience of the BBC Micro:Bit. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education* (Seattle, Washington, USA) (SIGCSE '17). Association for Computing Machinery, New York, NY, USA, 531–536. <https://doi.org/10.1145/3017680.3017749>
- [31] Markus Tyrén, Niklas Carlborg, Carl Heath, and Eva Eriksson. 2018. Considerations and Technical Pitfalls for Teaching Computational Thinking with BBC Micro:Bit. In *Proceedings of the Conference on Creativity and Making in Education* (Trondheim, Norway) (FabLearn Europe '18). Association for Computing Machinery, New York, NY, USA, 81–86. <https://doi.org/10.1145/3213818.3213829>
- [32] David Weintrop. 2019. Block-Based Programming in Computer Science Education. *Commun. ACM* 62, 8 (jul 2019), 22–25. <https://doi.org/10.1145/3341221>
- [33] David Weintrop and Uri Wilensky. 2015. To Block or Not to Block, That is the Question: Students' Perceptions of Blocks-Based Programming. In *Proceedings of the 14th International Conference on Interaction Design and Children* (Boston, Massachusetts) (IDC '15). Association for Computing Machinery, New York, NY, USA, 199–208. <https://doi.org/10.1145/2771839.2771860>
- [34] David Weintrop and Uri Wilensky. 2017. Comparing Block-Based and Text-Based Programming in High School Computer Science Classrooms. *ACM Trans. Comput. Educ.* 18, 1, Article 3 (oct 2017), 25 pages. <https://doi.org/10.1145/3089799>